

jDMRgrid: calling differentially methylated regions from WGBS data

Robert Stefan Piecyk, Rashmi Rekha Hazarika, Yadollah Shahryary Dizaji and Frank Johannes*

*robert.s.piecyk@gmail.com

2026-08-30

Contents

1	Overview.	2
2	A worked example	2
2.1	Call regions	3
2.2	Build the matrix.	4
2.3	Split into comparisons	5
2.4	Filter	5
2.5	Requiring a minimum posterior	6
2.6	Rank what survives.	7
2.7	Classify and annotate.	7
3	Input files	8
4	Files the pipeline writes	8
5	Choosing how regions are defined	9
6	Population data without replicates	10
7	Working from Bismark CX reports	11
8	Scale.	12
9	Troubleshooting.	12
10	Session info	13

1 Overview

jDMRgrid identifies differentially methylated regions (DMRs) between groups of samples in whole-genome bisulphite sequencing (WGBS) data.

The method proceeds in two stages. The genome is first partitioned into candidate **regions**, and a hidden Markov model is fitted to each region independently for every sample and sequence context, assigning a discrete **methylation state**: `0` for unmethylated, `1` for methylated and, where `include.intermediate=TRUE`, `0.5` for an intermediate state. The per-sample calls are then assembled into a **matrix** with one row per region and one column per sample. A region is reported as differentially methylated where the assigned state differs between groups.

Two properties of this design constrain what can be detected, and should be considered when interpreting the output.

- Detection requires a change of **state**, not a change of level. CG methylation declining from 75% to 55% is biologically substantial, but both values are classified as methylated, so no call is made. jDMRgrid reports regions that cross a state boundary.
- Each state call carries a posterior probability, unused during filtering and ranking unless requested. A call assigned at posterior 0.56 otherwise counts the same as one assigned at 0.99 (see *Requiring a minimum posterior*).

jDMRgrid supports two experimental designs, and the choice determines which functions apply and which filter is used.

design	sample sheet	comparison	filter
control/treatment with replicates	<code>file, sample, replicate, group</code>	pairwise between groups	<code>repligate.consensus</code>
population without replicates	<code>file, sample</code>	across all samples	<code>epiMAF.cutoff</code>

A *worked example* follows the control/treatment design from region calling to annotation; *Population data without replicates* covers the other. Region definition, confidence filtering, ranking, annotation and everything in *Scale* apply equally to both.

The pipeline comprises five functions, each operating on the output of the previous stage:

```
runjDMRgrid()    region calls, per sample, per context
makeDMRmatrix() one matrix, all samples
splitGroups()   one matrix per pairwise comparison (control/treatment only)
filterDMRmatrix() drop regions that are not differential
rankDMRs()      order what survives
```

`context.specific.DMRs()` and `annotatedDMRs()` then classify and annotate the result.

2 A worked example

The package includes the first 250 kb of *A. thaliana* chromosome 1 (91,971 cytosines) for four samples: two replicates of a wild type and two of a mutant. Sequencing depth averages 11.5 reads per cytosine with 3% of positions uncovered. The files were generated by running METHimpute over this interval, so posteriors and state calls are genuine, not placeholders. 99.9% of calls carry a posterior above 0.9.

jDMRgrid: calling differentially methylated regions from WGBS data

This governs the results below. The calls are well supported and the two groups agree across most of the interval, so **few regions survive filtering**: 1 in CG, 18 in CHG and 30 in CHH after overlapping windows are merged. That is the expected outcome for a well-powered comparison of two similar methylomes. Sparse coverage would yield many more apparent DMRs, most of them state calls fluctuating at random between samples.

This section follows the control/treatment design from region calling through to annotation. Population data without replicates constitutes a different design and is treated separately in *Population data without replicates*.

The sample sheet needs `file`, `sample`, `replicate` and `group`:

```
load(system.file("data", "listFiles2.RData", package="jDMRgrid"))
listFiles2
```

	file	sample	replicate	group
	<char>	<char>	<char>	<char>
1:	toyData/methimpute_p1.txt.gz	WT	rep1	control
2:	toyData/methimpute_p2.txt.gz	WT	rep2	control
3:	toyData/methimpute_p3.txt.gz	mutant1	rep1	treatment1
4:	toyData/methimpute_p4.txt.gz	mutant1	rep2	treatment1

2.1 Call regions

`runjDMRgrid()` tiles the genome into overlapping windows and fits the HMM in each. `window` and `step` set the tiling: equal values give non-overlapping bins, and a `step` below `window` slides them. `min.C` requires a minimum number of cytosines per window, so that states are not fitted on almost no data.

Complete argument documentation is given in `?runjDMRgrid`; the discussion below is confined to the consequences of each choice.

```
runjDMRgrid(
  out.dir = file.path(vig.dir, "replicate/grid"),
  window = 200, step = 50, samplelist = listFiles2,
  contexts = c("CG", "CHG", "CHH"), min.C = 10, min.C.type = "percentile",
  mincov = 0, include.intermediate = FALSE, runName = "Arabidopsis")
```

`window` and `step` are the principal parameters. A smaller window localises a DMR more precisely but places fewer cytosines in each region, so states are fitted on less evidence and posteriors decline; a larger window is better supported but blurs region boundaries and may average a short DMR against its surroundings. A 200 bp window with a 50 bp step is a reasonable starting point for *A. thaliana*; species with sparser methylation generally require larger windows.

`min.C.type` has no default and must be stated. Under `"percentile"`, `min.C=10` discards the least cytosine-dense tenth of windows; under `"count"` the same value requires ten cytosines per window. The two interpretations differ by orders of magnitude, and the meaning of `min.C` changed between versions of the package, so no default is applied.

The distinction also matters where the grid extends beyond the data. Chromosome lengths are inferred from the input, so a run restricted to part of a chromosome still tiles from the first position; if most windows contain no cytosines, a low percentile evaluates to zero, every empty window is retained and the HMM has nothing to fit.

jDMRgrid: calling differentially methylated regions from WGBS data

One file is written per sample per context:

```
region.calls <- fread(file.path(vig.dir, "replicate/grid/methimpute_p1-CG.txt"))
```

Table 2: Region calls for one sample, CG context.

seqnames	start	end	context	posteriorMax	status	rc.meth.lvl
1	1	200	CG	1.0000	M	0.7040
1	51	250	CG	1.0000	M	0.7040
1	101	300	CG	1.0000	M	0.7040
1	151	350	CG	1.0000	M	0.7040
1	301	500	CG	0.9999	U	0.0095
1	351	550	CG	1.0000	U	0.0094

`status` gives the state call and `rc.meth.lvl` a recalibrated methylation level derived from the posteriors and the fitted model parameters. Note that `rc.meth.lvl` is **not** the observed methylation of the region: it is a posterior-weighted mixture of the fitted state levels and therefore clusters around a small number of values. It is suitable for comparing regions within a single run, but should not be interpreted as a measurement of methylation.

2.2 Build the matrix

```
makeDMRmatrix(  
  contexts = c("CG", "CHG", "CHH"), postMax.out = TRUE,  
  samplelist = listFiles2,  
  input.dir = file.path(vig.dir, "replicate/grid"),  
  out.dir = file.path(vig.dir, "replicate/matrix"), include.intermediate = FALSE)
```

`include.intermediate` must match between `runjDMRgrid()` and `makeDMRmatrix()`. Calling regions with a three-state model and building the matrix with a two-state one leaves posteriors from a model the matrix does not describe. A uniform three-state posterior is 0.333, which reads like a low-confidence two-state call and is not one. Nothing warns about the mismatch.

`postMax.out=TRUE` is the default. It increases output volume by approximately 35-40%, but both `filterDMRmatrix(min.posterior=)` and `rankDMRs()` require the matrix it writes, so it is enabled by default so those options have input. Disable it only where storage is constrained and the posteriors are certainly not required.

```
statecalls <- fread(file.path(vig.dir, "replicate/matrix/CG_StateCalls.txt"), header=TRUE)
```

Table 3: State calls: 0 unmethylated, 1 methylated.

seqnames	start	end	WT_rep1	WT_rep2	mutant1_rep1	mutant1_rep2
1	1	200	1	1	1	1
1	51	250	1	1	1	1
1	101	300	1	1	1	1
1	151	350	1	1	1	1
1	301	500	0	0	0	0
1	351	550	0	0	0	0

Every region appears in three matrices with identical rows in identical order: the state calls above, the methylation levels, and the posteriors.

```
postMax <- fread(file.path(vig.dir, "replicate/matrix/CG_postMax.txt"), header=TRUE)
```

Table 4: Posterior probability behind each state call.

seqnames	start	end	WT_rep1	WT_rep2	mutant1_rep1	mutant1_rep2
1	1	200	1.0000	1	1	1
1	51	250	1.0000	1	1	1
1	101	300	1.0000	1	1	1
1	151	350	1.0000	1	1	1
1	301	500	0.9999	1	1	1
1	351	550	1.0000	1	1	1

2.3 Split into comparisons

With a `group` column, the comparison is pairwise and the matrix has to be split before it can be filtered.

```
splitGroups(
  samplelist = listFiles2,
  input.dir = file.path(vig.dir, "replicate/matrix"),
  out.dir = file.path(vig.dir, "replicate/matrix"))
```

This step is not optional. Once a sample sheet carries a `group` column, `filterDMRmatrix()` searches for the per-comparison files written here and fails if they are absent; see *Troubleshooting*.

2.4 Filter

`filterDMRmatrix()` applies up to four independent filters. Only the first is unconditional; the remainder are enabled by supplying the corresponding argument.

argument	effect
<i>always on</i>	Drops non-polymorphic regions, where every sample is in the same state and nothing can be differential.
<code>epiMAF.cutoff</code>	Population data. Drops regions where the rarer state is carried by fewer than this fraction of samples.
<code>replicate.consensus</code>	Control/treatment with replicates. Keeps a region only if each group's replicates agree and the groups differ from one another.
<code>min.posterior</code>	Any design. Drops regions where any sample's state was called below this posterior.

`epiMAF.cutoff` and `replicate.consensus` address different questions and are not normally combined: the former is appropriate for a population without replicates, the latter for a control/treatment design. Both default to `NULL`, so a call supplying neither removes only non-polymorphic regions.

The value of `replicate.consensus` specifies the concordance required among the replicates of a group. With two replicates the only meaningful value is `1`.

jDMRgrid: calling differentially methylated regions from WGBS data

One threshold is **not** exposed. A region must additionally exhibit a between-group difference of at least `diff.ct` in the proportion of replicates called methylated, and this is fixed internally at **0.5**. With three replicates per group this requires at least a 2-of-3 versus 0-of-3 split, and cannot be relaxed from the top-level call.

`if.mergingBins` merges consecutive bins carrying identical calls across all samples, turning a run of overlapping windows into a single region. **It defaults to TRUE**, and there are few circumstances in which it should be disabled.

With `window=200` and `step=50` the grid advances in 50 bp increments, so a single differential region is spanned by several overlapping windows, each reported as a separate row. In the packaged data, 56 unfiltered CHH rows correspond to only 30 distinct intervals, and one region (103,106-104,298) accounts for 16 of them. Without merging, this propagates through the remainder of the analysis: a ranked list is dominated by near-duplicates of a single region, and any count of DMRs reflects the geometry of the window grid, not the biology.

```
filterDMRmatrix(  
  epiMAF.cutoff = NULL, replicate.consensus = 0.8,  
  data.dir = file.path(vig.dir, "replicate/matrix"),  
  samplelist = listFiles2, if.mergingBins = TRUE)
```

Because the sheet has groups, the filtered output carries the comparison in its name:

```
## CHH carries most of the difference between these two groups; CG barely differs at all  
filtered <- fread(file.path(vig.dir,  
  "replicate/matrix/CHH_WT_mutant1_StateCalls-filtered.txt"), header=TRUE)
```

Table 6: Regions surviving the filters.

seqnames	start	end	WT_rep1	WT_rep2	mutant1_rep1	mutant1_rep2
1	23851	24100	1	1	0	0
1	29251	29500	1	1	0	0
1	38751	38950	0	0	1	1
1	39451	39800	1	1	0	0
1	57001	57200	1	1	0	0
1	58501	58700	1	1	0	0

2.5 Requiring a minimum posterior

Nothing so far has used the posteriors. `min.posterior` keeps only regions where every sample in the comparison was called with at least that confidence.

```
conf.dir <- file.path(vig.dir, "confidence")  
dir.create(conf.dir, showWarnings = FALSE)  
invisible(file.copy(  
  list.files(file.path(vig.dir, "replicate/matrix"), full.names = TRUE),  
  conf.dir, overwrite = TRUE))  
filterDMRmatrix(  
  replicate.consensus = 0.8, data.dir = conf.dir,  
  samplelist = listFiles2, min.posterior = 0.9, if.mergingBins = TRUE)
```

0.9 is the usual threshold and is what this example uses. It removes little here because the packaged data is well covered, but on a dataset with thinner coverage it removed about half the regions that survived the other filters.

2.6 Rank what survives

Filtering returns an unordered set in which every surviving region is equally a DMR. `rankDMRs()` imposes an ordering by one of three methods.

- `effect`: absolute difference in methylation between groups, weighted by the posterior.
- `betabinom`: a Wald statistic with a beta-binomial variance and a context-specific overdispersion `rho`. The usual default.
- `permutation`: an empirical null from permuted group labels, with BH-adjusted FDR. This one needs replicates. With three against three only 20 distinct label assignments exist, and `rankDMRs()` warns when the null is too small for the FDR to mean anything.

Effect sizes are recomputed from the input files as observed methylation, defined as `sum(methylated)/sum(total)` per region. Specifying `level.from="states"` uses `rc.meth.lvl` instead, which, as noted above, is a mixture of fitted state levels and cannot represent a change confined to a single state.

```
ranked.dir <- file.path(vig.dir, "ranked")
rankDMRs(
  data.dir = file.path(vig.dir, "replicate/matrix"), samplefiles = listFiles2,
  contexts = "CHH", method = "betabinom", out.dir = ranked.dir, if.filtered = TRUE)
```

```
## the ranked table is wide; these are the columns worth reading first
knitr::kable(
  head(fread(file.path(ranked.dir, "CHH_rankedDMRs.txt"))[
    , .(seqnames, start, end, delta, min_posterior, se, stat, rank)], 4),
  digits = 4, caption = "Top-ranked DMRs (betabinom).")
```

Table 7: Top-ranked DMRs (betabinom).

seqnames	start	end	delta	min_posterior	se	stat	rank
1	103101	103500	0.1633	0.9720	0.0788	2.0724	1
1	103551	104450	0.1337	0.9674	0.0719	1.8604	2
1	212901	213150	-0.0094	0.8092	0.0256	0.3686	3
1	76001	76200	-0.0099	0.6889	0.0293	0.3369	4

2.7 Classify and annotate

`context.specific.DMRs()` separates CG-only, CHG-only, CHH-only, non-CG and multi-context DMRs. `if.filtered` selects whether it reads the filtered matrices or the unfiltered ones.

```
context.specific.DMRs(
  samplelist = listFiles2,
  output.dir = file.path(vig.dir, "replicate/context_DMRs"),
  input.dir = file.path(vig.dir, "replicate/matrix"), if.filtered = FALSE)
```

`annotatedDMRs()` maps any three-column file (chromosome, start, end) against one or more GFF3 annotations. Put the files to annotate in their own directory and point `input.dir` at it.

```
gff.file_promoters <- system.file(
  "extdata/toyData", "TAIR10_promoters.gff3", package="jDMRgrid")
gff.file_TE <- system.file(
  "extdata/toyData", "TAIR10_TE.gff3", package="jDMRgrid")
```

jDMRgrid: calling differentially methylated regions from WGBS data

```
annotatedDMRs(  
  gff.files = c(gff.file_promoters, gff.file_TE),  
  annotation = c("promoters", "TE"),  
  input.dir = file.path(vig.dir, "replicate/context_DMRs"), if.gff3 = FALSE,  
  out.dir = file.path(vig.dir, "replicate/annotate_DMRs"))
```

```
DMRcounts <- fread(  
  file.path(vig.dir, "replicate/annotate_DMRs/DMR-counts.txt"), header=TRUE)
```

Table 8: DMR counts per annotation category.

sample	total.DMRs	promoters	TE	multiple.overlaps
WT_mutant1_nonCG-DMRs	10	3	0	2

3 Input files

jDMRgrid accepts base-level methylation in either of two forms, and everything downstream of the region calls is identical for both.

METHimpute output is the default. These files carry every cytosine in the genome, including uncovered positions, together with its context.

Bismark cytosine (CX) reports are read when `if.Bismark=TRUE`, and then a reference FASTA is required as well. A CX report lists only the covered cytosines, so jDMRgrid reconstructs the full cytosine set from the genome and inflates the report onto it. See *Working from Bismark CX reports*.

The sample sheet is a data frame. `file` and `sample` are always required; `replicate` and `group` are required for a control/treatment design and omitted for population data.

4 Files the pipeline writes

Each stage reads the directory the previous stage wrote, so the arguments chain: `runjDMRgrid(out.dir=)` becomes `makeDMRmatrix(input.dir=)`, whose `out.dir` becomes `filterDMRmatrix(data.dir=)` and `rankDMRs(data.dir=)`. The filenames matter, because several functions locate their inputs by name.

`runjDMRgrid()` writes, into `out.dir`:

- `<sample>_<context>.txt`, the region calls, one file per sample per context
- the region definition itself, as `<runName>_Win<window>_Step<step>_<context>.Rdata`
- `<runName>_optimal_minC_threshold.csv`

`makeDMRmatrix()` writes, into `out.dir`:

- `<context>_StateCalls.txt` and `<context>_rcMethlvl.txt`
- `<context>_postMax.txt`, only when `postMax.out=TRUE`

`splitGroups()` applies to the control/treatment design only. It rewrites those matrices per pairwise comparison, with the matching `rcMethlvl` file alongside each:

jDMRgrid: calling differentially methylated regions from WGBS data

```
<context>_<group1>_<group2>_StateCalls.txt
```

Population data has no groups to split, so its matrices keep the plain `<context>_` names throughout.

`filterDMRmatrix()` writes the same names with a `-filtered` suffix, for example `<context>_StateCalls-filtered.txt`.

`rankDMRs()` writes `<context>_rankedDMRs.txt`.

`context.specific.DMRs()` writes one file per category: CG-only, CHG-only, CHH-only, non-CG and multi-context.

`annotatedDMRs()` writes `<comparison>_annotation.txt` per comparison, plus a single `DMR-counts.txt` summary.

The state calls and the methylation levels are always written as a pair, with identical regions in identical order. Anything reading one should match the other by coordinate rather than by row position, because the filtered variants are subsets.

5 Choosing how regions are defined

The candidate regions scored by `runjDMRgrid()` can be defined in three ways, selected with `region.mode`. This is independent of the experimental design and applies to population and control/treatment data alike. Everything downstream is unchanged.

region.mode	candidate regions
"grid" (default)	fixed windows tiled uniformly across the whole genome
"cluster"	cytosines of each context are clustered, and the clusters are the regions
"cluster-grid"	cytosines are clustered first, then windows are tiled inside clusters only

Under the two clustering modes, adjacent cytosines are merged whenever the gap between them is below `max.gap`, and clusters holding fewer than `min.sites` cytosines are dropped. `max.gap="adaptive"` picks the gap per context from the observed distribution of inter-cytosine distances, at the quantile given by `gap.quantile`. This matters for dense contexts: at a fixed gap, CHH tends to merge into chromosome-scale blocks.

```
runjDMRgrid(  
  out.dir = file.path(vig.dir, "clustered"), window = 200, step = 50,  
  samplelist = listFiles2, contexts = "CG", min.C = 5, mincov = 0,  
  include.intermediate = FALSE, runName = "clustered", min.C.type = "count",  
  region.mode = "cluster", max.gap = "adaptive", min.sites = 5)
```

On the packaged toy data the three modes give very different region counts, because the grid covers the whole 250 kb slice while clusters cover only its cytosine-bearing parts:

The grid remains the default. On a whole genome, `"cluster-grid"` was measured to produce 76% more regions than the grid and to run 74% more slowly. It should therefore be selected when variable-density regions are specifically required, and not as a means of reducing runtime.

Table 10: Regions called for CG on the packaged toy data.

region.mode	regions
grid	3922
cluster	702
cluster-grid	1187

6 Population data without replicates

A population without replicates is a different design: there are no groups to compare, so `splitGroups()` does not apply and the filter is minor epi-allele frequency, not replicate consensus. The sheet needs only `file` and `sample`.

```
load(system.file("data", "listFiles1.RData", package="jDMRgrid"))
listFiles1
```

```

      file      sample replicate
      <char>    <char>    <char>
1: toyData/methimpute_p1.txt.gz methylomeA      repl
2: toyData/methimpute_p2.txt.gz methylomeB      repl
3: toyData/methimpute_p3.txt.gz methylomeC      repl
4: toyData/methimpute_p4.txt.gz methylomeD      repl
```

```

runjDMRgrid(
  out.dir = file.path(vig.dir, "population/grid"),
  window = 200, step = 50, samplelist = listFiles1,
  contexts = c("CG", "CHG", "CHH"), min.C = 10, min.C.type = "percentile",
  mincov = 0, include.intermediate = FALSE, runName = "Arabidopsis")
makeDMRmatrix(
  contexts = c("CG", "CHG", "CHH"), postMax.out = TRUE,
  samplelist = listFiles1,
  input.dir = file.path(vig.dir, "population/grid"),
  out.dir = file.path(vig.dir, "population/matrix"), include.intermediate = FALSE)
filterDMRmatrix(
  epiMAF.cutoff = 0.33, replicate.consensus = NULL,
  data.dir = file.path(vig.dir, "population/matrix"),
  samplelist = listFiles1, if.mergingBins = TRUE)
```

Without a `group` column the filtered files keep the plain names:

```

pop.filtered <- fread(
  file.path(vig.dir, "population/matrix/CG_StateCalls-filtered.txt"), header=TRUE)
```

Table 11: Population DMRs after epiMAF filtering.

seqnames	start	end	methylomeA_rep1	methylomeB_rep1	methylomeC_rep1	methylomeD_rep1
1	29651	29850	0	1	0	0
1	34301	34500	1	0	0	0
1	34401	34600	1	0	1	1
1	35051	35250	0	1	0	0
1	35651	35850	0	0	0	1
1	36251	36600	0	1	0	0

Ranking, context-specific classification and annotation then work exactly as in the worked example, reading the plain `<context>` matrices. `rankDMRs()` needs two groups to compare, so it applies to the control/treatment design only; the remaining stages are common to both.

7 Working from Bismark CX reports

A CX report is the usual seven-column Bismark format: chromosome, position, strand, count methylated, count unmethylated, context, and trinucleotide context.

```
cx.dir <- system.file("extdata", "toyData", package = "jDMRgrid")
cx.head <- read.delim(gzfile(file.path(cx.dir, "cxSample1.CX_report.txt.gz")),
                     header = FALSE, nrows = 3)
colnames(cx.head) <- c("chr", "pos", "strand", "meth", "unmeth", "context", "trinuc")
knitr::kable(cx.head, caption = "First rows of a Bismark CX report.")
```

Table 12: First rows of a Bismark CX report.

chr	pos	strand	meth	unmeth	context	trinuc
1	1	+	0	0	CHH	CCC
1	2	+	0	0	CHH	CCT
1	3	+	0	0	CHH	CTA

The package ships two small CX reports covering the first 50 kb of *A. thaliana* chromosome 1, together with the matching genome slice, so this section runs in a few seconds.

```
cxlist <- data.frame(
  file      = file.path(cx.dir, c("cxSample1.CX_report.txt.gz",
                                "cxSample2.CX_report.txt.gz")),
  sample    = c("cxA", "cxB"),
  replicate = c("rep1", "rep1"),
  group     = c("control", "treatment"),
  stringsAsFactors = FALSE)

## shown with basenames; the pipeline uses the full paths above
transform(cxlist, file = basename(file))
```

	file	sample	replicate	group
1	cxSample1.CX_report.txt.gz	cxA	rep1	control
2	cxSample2.CX_report.txt.gz	cxB	rep1	treatment

The only differences from the METHimpute route are `if.Bismark=TRUE` and `FASTA.file`. Everything downstream is identical.

```
cx.out <- file.path(vig.dir, "bismark")
runjDMRgrid(
  out.dir = file.path(cx.out, "grid"), window = 200, step = 50,
  samplelist = cxlist, contexts = "CG", min.C = 5, mincov = 0,
  include.intermediate = FALSE, runName = "cx",
  if.Bismark = TRUE, FASTA.file = file.path(cx.dir, "chr1_50kb.fa"),
  min.C.type = "count")
makeDMRmatrix(
  contexts = "CG", postMax.out = TRUE, samplelist = cxlist,
```

jDMRgrid: calling differentially methylated regions from WGBS data

```
input.dir = file.path(cx.out, "grid"), out.dir = file.path(cx.out, "matrix"),
include.intermediate = FALSE, if.Bismark = TRUE)
```

```
knitr::kable(fread(file.path(cx.out, "matrix", "CG_StateCalls.txt"))[1:4],
caption = "DMR matrix built from Bismark CX reports.")
```

Table 13: DMR matrix built from Bismark CX reports.

seqnames	start	end	cxA_rep1	cxB_rep1
1	1	200	1	1
1	51	250	1	1
1	101	300	1	1
1	451	650	1	1

8 Scale

The dataset in this vignette is a 250 kb slice and runs in well under a minute. Real genomes do not, and the cost is concentrated in one place: **region calling accounts for roughly 78% of total runtime**, with the DMR matrix, filtering and annotation sharing the remainder.

`numCores` parallelises region calling, but the returns fall away quickly. Measured on a full Arabidopsis dataset:

cores	runtime	speed-up	efficiency
1	4465 s	1.0x	100%
2	2481 s	1.8x	90%
4	1429 s	3.1x	78%
8	928 s	4.8x	60%
16	669 s	6.7x	42%
32	650 s	6.9x	22%

Beyond 8 cores the computation is limited by memory bandwidth, not processor time, and 32 cores confers little advantage over 16. Eight is a reasonable default.

Two further factors govern cost. CHH contains substantially more cytosines than CG or CHG, so a three-context run is dominated by CHH; restricting `contexts` to those actually required is the single most effective economy. Additionally, `region.mode="cluster-grid"` was measured to generate 76% more regions than the grid and to run 74% more slowly, and is therefore not a means of reducing runtime.

9 Troubleshooting

"Files do not exist or is non-readable!" from `filterDMRmatrix()`. The sample sheet contains a `group` column, so the function searches for the per-comparison matrices written by `splitGroups()` and does not find them. Run `splitGroups()` beforehand. Population sheets carry no `group` column and are unaffected. The message identifies no file and makes no reference to `splitGroups()`, and is therefore easily misread as a failure of `makeDMRmatrix()`.

A single-context run cannot be filtered. No error is raised for `contexts="CG"` until the filtering stage, which operates on the contexts present in the matrix directory. Running a single context is the most efficient way to test a configuration, but the full set should be generated before filtering.

`min.posterior` or `rankDMRs()` reports no input. Both require the posterior matrix. If `makeDMRmatrix()` was called with `postMax.out=FALSE`, it must be re-run with the default `TRUE`.

“no regions survived for context ...”. `min.C` is too stringent for the data. Note that `min.C.type` determines whether the value is read as a percentile or an absolute count; the same number carries very different meanings under the two settings.

A clustering mode collapses into a small number of very large regions. At a fixed `max.gap`, dense contexts merge into chromosome-scale blocks. Specifying `max.gap="adaptive"` derives the gap from the data separately for each context.

10 Session info

```
sessionInfo()
```

```
R version 4.4.2 (2024-10-31)
Platform: x86_64-pc-linux-gnu
Running under: Ubuntu 22.04.5 LTS

Matrix products: default
BLAS:   /usr/lib/x86_64-linux-gnu/blas/libblas.so.3.10.0
LAPACK: /usr/lib/x86_64-linux-gnu/lapack/liblapack.so.3.10.0

locale:
 [1] LC_CTYPE=en_US.UTF-8      LC_NUMERIC=C
 [3] LC_TIME=en_US.UTF-8      LC_COLLATE=en_US.UTF-8
 [5] LC_MONETARY=en_US.UTF-8  LC_MESSAGES=en_US.UTF-8
 [7] LC_PAPER=en_US.UTF-8     LC_NAME=C
 [9] LC_ADDRESS=C             LC_TELEPHONE=C
[11] LC_MEASUREMENT=en_US.UTF-8 LC_IDENTIFICATION=C

time zone: Etc/UTC
tzcode source: system (glibc)

attached base packages:
[1] stats      graphics  grDevices  utils      datasets  methods    base

other attached packages:
[1] data.table_1.18.0 jDMRgrid_0.3.0   BiocStyle_2.34.0

loaded via a namespace (and not attached):
 [1] tidyselect_1.2.1      dplyr_1.1.4
 [3] farver_2.1.2          R.utils_2.13.0
 [5] Biostrings_2.74.1     S7_0.2.1
 [7] bitops_1.0-9          fastmap_1.2.0
```

jDMRgrid: calling differentially methylated regions from WGBS data

[9] RCurl_1.98-1.17	GenomicAlignments_1.42.0
[11] XML_3.99-0.20	digest_0.6.39
[13] lifecycle_1.0.4	magrittr_2.0.4
[15] compiler_4.4.2	rlang_1.1.6
[17] tools_4.4.2	yaml_2.3.12
[19] rtracklayer_1.66.0	knitr_1.51
[21] S4Arrays_1.6.0	curl_7.0.0
[23] DelayedArray_0.32.0	plyr_1.8.9
[25] RColorBrewer_1.1-3	abind_1.4-8
[27] BiocParallel_1.40.2	withr_3.0.2
[29] purrr_1.2.0	R.oo_1.27.1
[31] BiocGenerics_0.52.0	grid_4.4.2
[33] stats4_4.4.2	future_1.68.0
[35] ggplot2_4.0.1	globals_0.18.0
[37] scales_1.4.0	iterators_1.0.14
[39] SummarizedExperiment_1.36.0	cli_3.6.5
[41] rmarkdown_2.30	crayon_1.5.3
[43] generics_0.1.3	otel_0.2.0
[45] future.apply_1.20.1	httr_1.4.7
[47] reshape2_1.4.4	rjson_0.2.23
[49] RUnit_0.4.33.1	ape_5.8-1
[51] stringr_1.6.0	zlibbioc_1.52.0
[53] parallel_4.4.2	BiocManager_1.30.27
[55] XVector_0.46.0	restfulr_0.0.16
[57] matrixStats_1.5.0	vctrs_0.6.5
[59] Matrix_1.7-4	jsonlite_2.0.0
[61] minpack.lm_1.2-4	bookdown_0.46
[63] IRanges_2.40.1	S4Vectors_0.44.0
[65] RBGL_1.82.0	listenv_0.10.0
[67] foreach_1.5.2	tidyr_1.3.2
[69] glue_1.8.0	parallelly_1.46.0
[71] codetools_0.2-20	stringi_1.8.7
[73] gtable_0.3.6	GenomeInfoDb_1.42.3
[75] GenomicRanges_1.58.0	BiocIO_1.16.0
[77] UCSC.utils_1.2.0	tibble_3.3.0
[79] pillar_1.11.1	htmltools_0.5.9
[81] graph_1.84.1	methimpute_1.28.0
[83] GenomeInfoDbData_1.2.13	R6_2.6.1
[85] doParallel_1.0.17	lattice_0.22-7
[87] evaluate_1.0.5	Biobase_2.66.0
[89] R.methodsS3_1.8.2	Rsamtools_2.22.0
[91] Rcpp_1.1.0	nlme_3.1-168
[93] SparseArray_1.6.2	xfun_0.55
[95] MatrixGenerics_1.18.1	biocViews_1.74.0
[97] pkgconfig_2.0.3	